

Day 3: Change Password

1. Day Number

Day 3 of the Python foundation plan

2. Topic Name

Nested if condition

Today you will learn how to:

- Place one decision inside another decision
 - Validate the current password before accepting a new password
 - Reassign a variable
 - Reject an empty new password
-

3. Connection with Previous Days

On Day 1, you stored a password.

On Day 2, you compared an entered password with a stored password.

Today, you will use that comparison before allowing a password change. Only a user who enters the correct current password should be allowed to provide a new one.

After that first decision succeeds, a second decision checks whether the new password is empty.

4. Important Topics

Nested if

A nested if is an if statement inside another if block.

```
if first_condition:
    if second_condition:
        print("Both conditions are true")
```

The inner condition is checked only when the outer condition is true.

Reassignment

Reassignment replaces the value currently stored in a variable.

```
status = "old"
status = "new"
```

After the second statement, status contains "new".

String comparison

Use == to check whether the current password matches the stored password.

Non-empty validation

An empty string looks like this:

```
" "
```

A simple check for a non-empty value is:

```
value != ""
```

Indentation

The inner decision must be indented inside the outer decision. Its actions need another level of indentation.

5. Foundational Notes

Validate identity first

The program must check the current password before asking for or updating the new password.

The order should be:

```
Check current password
    Then check new password
        Then update
```

The inner condition depends on the outer condition

If the current password is wrong, Python should skip the entire new-password block.

Updating means assigning again

The stored password can be updated by assigning the new value to the existing variable.

Do not update before validation

The variable should remain unchanged when:

- The current password is wrong.
- The new password is empty.

Password comparison remains case-sensitive

Uppercase letters, lowercase letters, symbols, and spaces must match exactly.

Do not display a real password

Printing the updated password is unsafe. A success message is enough to confirm that the change occurred.

Scope for this beginner exercise

The changed password exists only while the program is running. Permanent storage will be introduced during the file-handling lessons.

6. Easy Example

Suppose a program changes a saved city only after a simple code is verified:

```
saved_code = "1234"
entered_code = input("Enter code: ")

if entered_code == saved_code:
    new_city = input("Enter new city: ")
    if new_city != "":
        city = new_city
        print("City updated.")
    else:
        print("City cannot be empty.")
else:
    print("Wrong code.")
```

This demonstrates a nested decision and reassignment using non-password data. Use the same reasoning for today's task, but write the password program yourself.

7. Problem Statement

Create a beginner-friendly Python program that:

1. Stores an old password.
2. Asks the user to enter the current password.
3. Checks whether the current password is correct.
4. If it is correct, asks the user to enter a new password.
5. Updates the stored password only when the new password is not empty.
6. Prints an appropriate success or error message.
7. Does not display the stored or updated password.

Use a functional approach.

8. Concepts Used

- Variables
 - Strings
 - `input()` and `print()`
 - Assignment and reassignment
 - `==` and `!=`
 - Outer if-else
 - Nested if-else
 - Indentation
 - Validation
 - Function
-

9. Thought Process

Step 1: Store the existing password

The program needs an old password that represents the current saved value.

Step 2: Ask for the current password

Receive the user's verification attempt with `input()`.

Step 3: Make the outer decision

Compare the entered current password with the stored password.

Step 4: Reject an incorrect current password

When the comparison is false, print an error and do not ask for a new password.

Step 5: Ask for a new password

This step belongs inside the successful branch of the outer decision.

Step 6: Make the nested decision

Check whether the new password is empty.

Step 7: Update only a valid new password

When it is not empty, assign the new value to the stored-password variable and print a success message.

Step 8: Reject an empty new password

When the inner condition fails, print a friendly error and keep the original password.

10. Pseudocode

```
START

DEFINE a change-password function
    STORE the old password
    ASK the user for the current password

    IF current password equals old password
        ASK the user for a new password

        IF new password is not empty
            REASSIGN old password to the new password
            DISPLAY password-changed success message
        ELSE
            DISPLAY new-password-cannot-be-empty message
    ELSE
        DISPLAY wrong-current-password message

CALL the function

END
```

11. Suggested Solving Approach: Functional Approach

Create one function containing the complete change-password process.

Use:

- One stored-password variable
- Two calls to `input()`
- One outer `if-else`
- One inner `if-else`
- Reassignment only inside the successful inner branch

Use this nesting shape:

```
if first_condition:
    if second_condition:
        # Update
    else:
        # Handle invalid new value
else:
    # Handle failed verification
```

Fill in the conditions and messages from the problem statement.

12. Easy Edge Cases

Wrong old password

The program should print an error, skip the new-password question, and keep the original value.

Empty new password

If the user presses Enter without typing a new password, the program should reject the change.

Correct old password and valid new password

The program should reassign the stored-password variable and print a success message.

Uppercase or lowercase difference

An entered current password with different capitalization should not match.

New password containing spaces

For this exercise, a string containing spaces is not technically empty. More advanced password-strength validation can be added later.

13. Expected Input

Successful example:

```
Enter current password: hello123
Enter new password: python456
```

Wrong-current-password example:

Enter current password: wrong123

Empty-new-password example:

Enter current password: hello123
Enter new password:

14. Expected Output

For a correct current password and non-empty new password:

Password changed successfully.

For a wrong current password:

Wrong current password.

For an empty new password:

New password cannot be empty.

15. Hint Only

The new-password input must be inside the branch that handles a correct current password.

Then place a second if-else inside that branch:

```
if current_value == stored_value:
    # Ask for replacement value
    if replacement_value != "":
        # Reassign and confirm
    else:
        # Reject empty value
else:
    # Reject incorrect current value
```

Do not update the stored password in either error branch, and do not print the password.